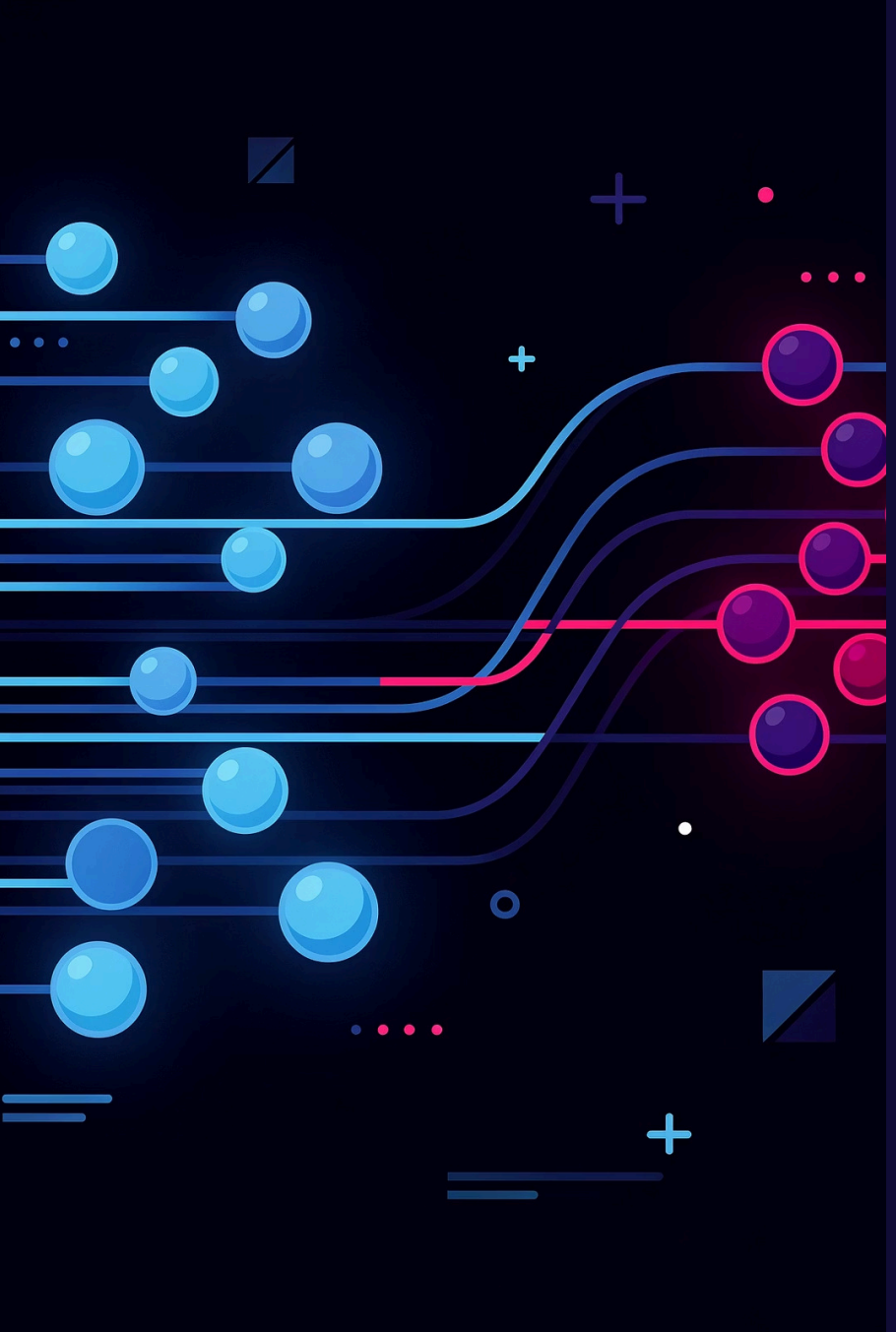


Applied GenAI Engineering – Session 7

Building RAG Systems from Scratch

Instructor: **Shrey** · Live Coding Session

Where We Are

Already Covered

- Embeddings — how text becomes vectors
- Vector databases — indexing and similarity search
- Retrieval pipeline — conceptual architecture

The Gap

We have discussed the theory. We have analysed the architecture. We have debated the trade-offs.

But we have NOT built the system yet.

Today we close that gap - from zero to a working RAG pipeline in pure Python.

Today's Objective

Build a fully working RAG pipeline from scratch - no frameworks, no magic, just Python and first principles.

- Ingestion
- Chunking
- Retrieval
- Generation

What is RAG?

Retrieval Augmented Generation

Retrieval

Find information relevant to the user's query from an external knowledge source

Augmented

Inject that information as context directly into the LLM prompt

Generation

LLM produces a grounded, factual answer based on the provided context

The Core Insight

LLMs are frozen in time. They know the world up to their training cutoff - and they know *nothing* about your data.

RAG gives them eyes. We retrieve the facts; the LLM reasons over them.

RAG = System Components

Every abstract concept in RAG maps directly to a concrete engineering component. There is no mystery - just code.

Retrieval

sentence-transformers + FAISS index

Augmentation

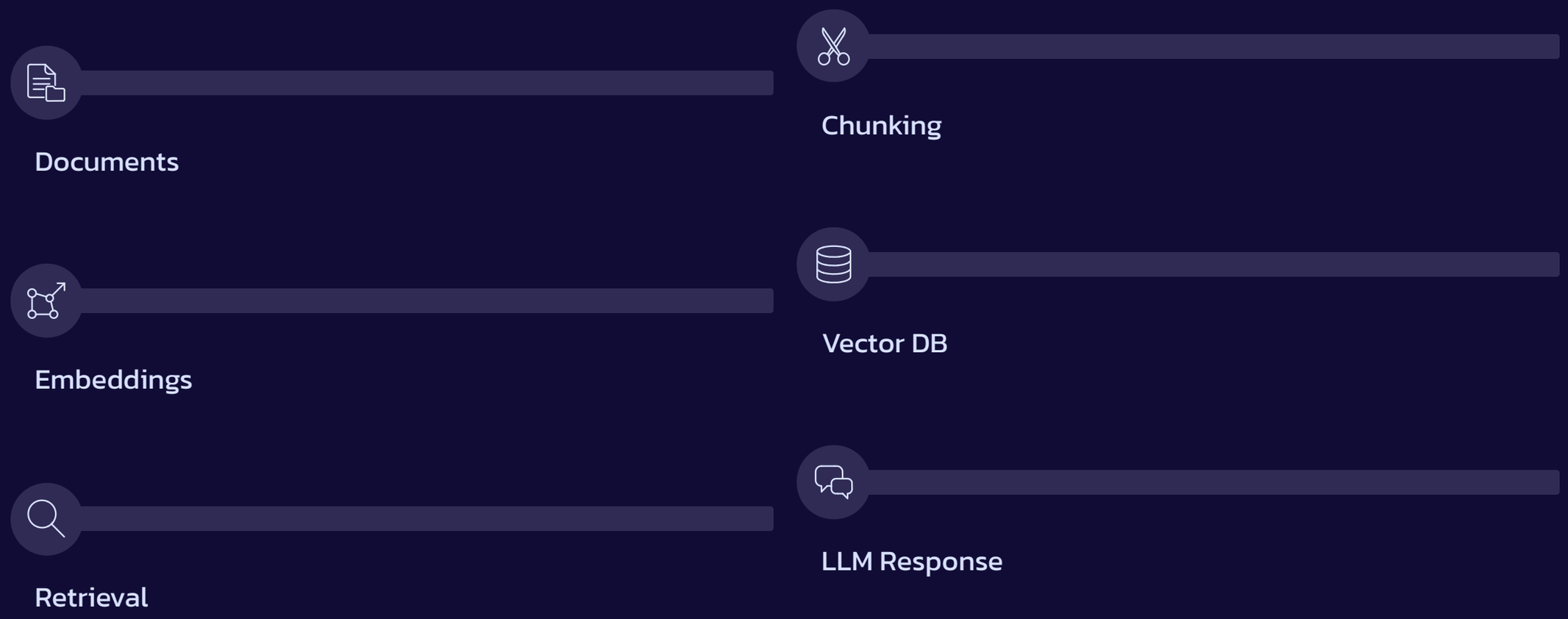
Retrieved chunks injected into prompt template

Generation

LLM API call with grounded context

Engineering mantra: Every part of RAG maps to code. If you can't point to it in the codebase, it doesn't exist.

RAG Architecture



The pipeline is strictly linear - data flows in one direction. Documents are chunked into segments, each segment is encoded into a dense vector, and those vectors are stored in FAISS (or any other vector DB). At query time, the query is embedded using the same model, the nearest vectors are retrieved, and the result is passed to the LLM as context.

Key Statement

RAG is a system, not a library. You wire together discrete components - and that's the point.

Mapping Architecture → Codebase

Here's how the abstract RAG architecture maps to concrete files in our `app/rag/` directory. Each component of the RAG pipeline has a dedicated, focused module.

`ingestion.py`

load documents

`chunking.py`

create chunks

`embedding.py`

MiniLM embeddings

`vector_store.py`

FAISS index

`retriever.py`

top-K retrieval

`prompt_builder.py`

context injection

`pipeline.py`

full RAG flow

`playground.py`

test runner

Two Pipelines

RAG has two distinct execution paths. Understanding this separation is essential for production reasoning.

OFFLINE — BUILD TIME

Indexing Pipeline

01

Load

Read documents from disk

02

Chunk

Split into fixed-size segments with overlap

03

Embed

Encode each chunk via MiniLM

04

Index

Store vectors in FAISS

ONLINE — QUERY TIME

Query Pipeline

01

Query

Receive user question

02

Embed

Encode query with same MiniLM model

03

Retrieve

FAISS returns top-K nearest chunks

04

Generate

LLM produces grounded answer

Dataset Design

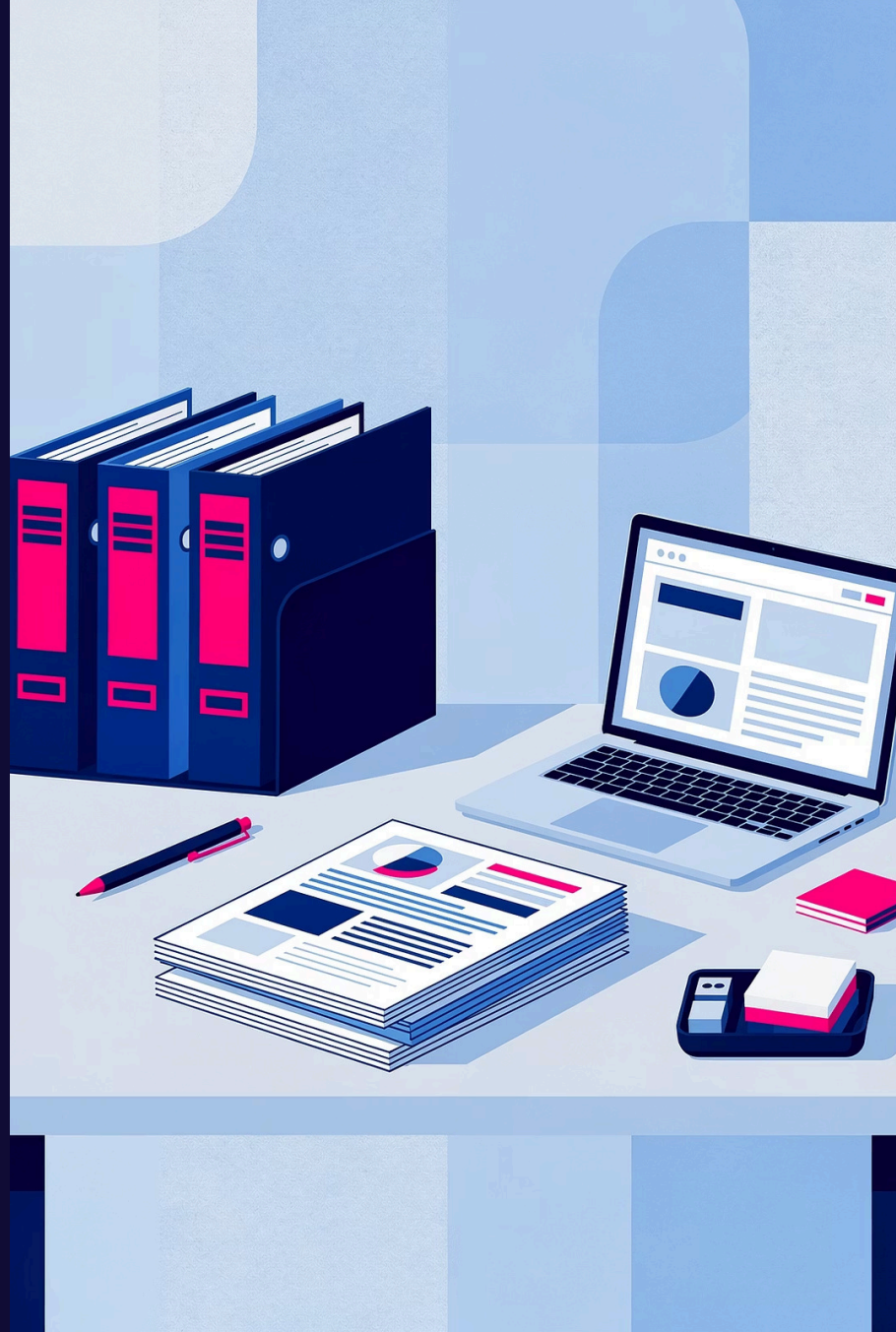
📁 **data/**

```
hr_policy_expanded.txt
product_manual_expanded.txt
reimbursement_policy.txt
security_guidelines.txt
onboarding_guide.txt
```

Why This Dataset?

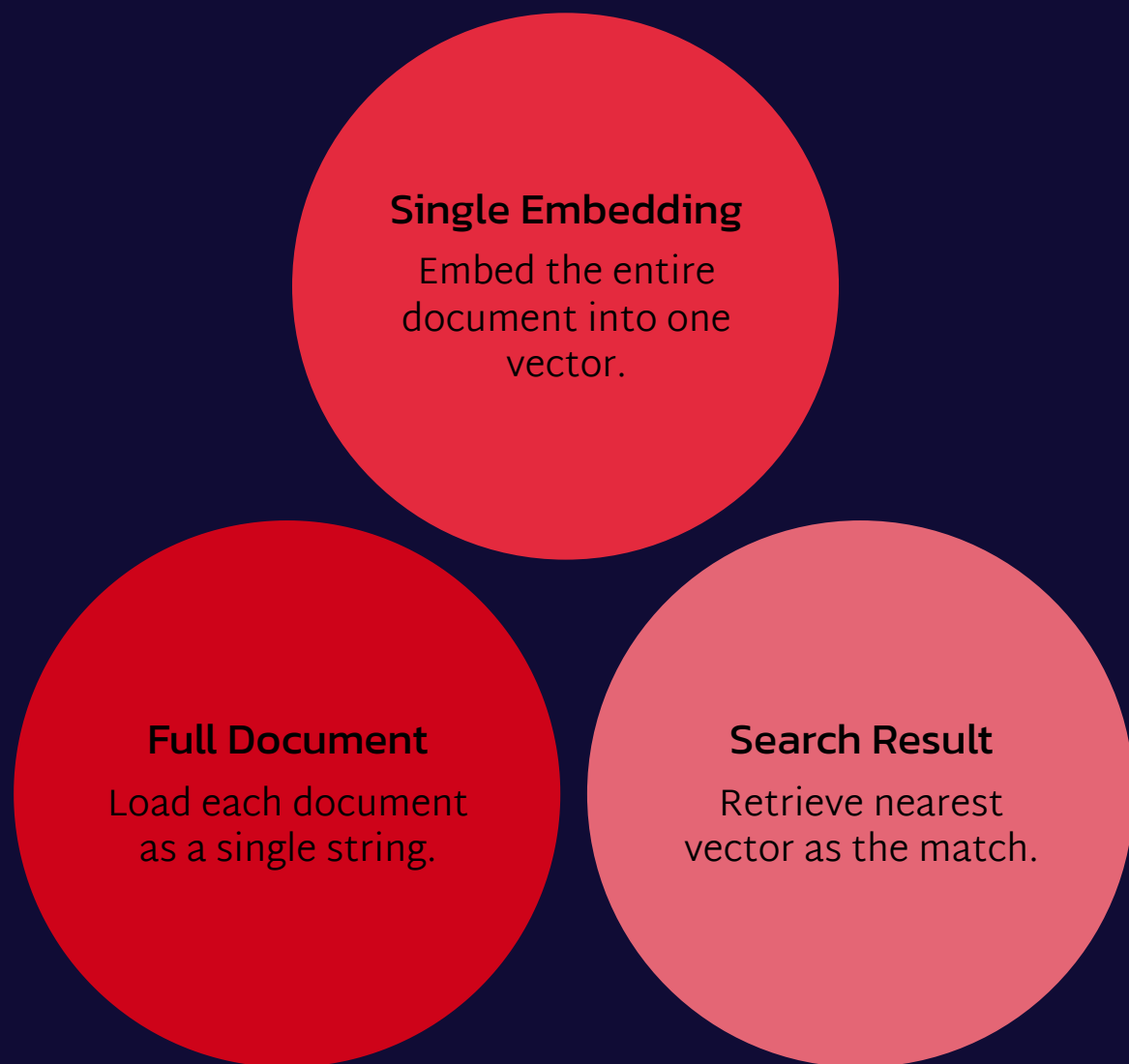
These five documents are deliberately chosen to **stress-test naive retrieval**. Topics overlap, terminology is shared, and a full-document retrieval approach will consistently return noisy, imprecise results.

- HR and onboarding content shares vocabulary
- Reimbursement spans policy and product domains
- Security guidelines bleed into onboarding flow



Naive Approach — No Chunking

Before we build the right system, let's build the wrong one. This is the first live coding exercise.



What We'll Code

- Load each document as a single string
- Embed the entire document with MiniLM
- Store one vector per document in FAISS
- Query and return the top-1 result

Think Before You Run

What could go wrong?

Before running the code, predict the failure mode. Engineering intuition is built by anticipating problems - not just debugging them.

Expected Failure

Run the naive pipeline. Observe the output. This is intentional.

Entire Documents Retrieved

Every query returns a full document - thousands of tokens of context passed to the LLM with no filtering.

Too Much Noise

A single document embedding averages all its topics into one vector - specific signals are drowned out

Poor Precision

The "right" document may rank correctly, but the wrong sections within it dominate the context window

Weak Answers

The LLM receives bloated, unfocused context - responses are vague, hedged, or factually muddled

Next step: Don't just observe the failure - diagnose the root cause before opening the fix.

Why It Failed

Root Cause

The retrieval unit is the full document.

When your atomic unit of retrieval is too large, precision collapses — regardless of how good your embedding model is.

Three Failure Modes

No Granularity

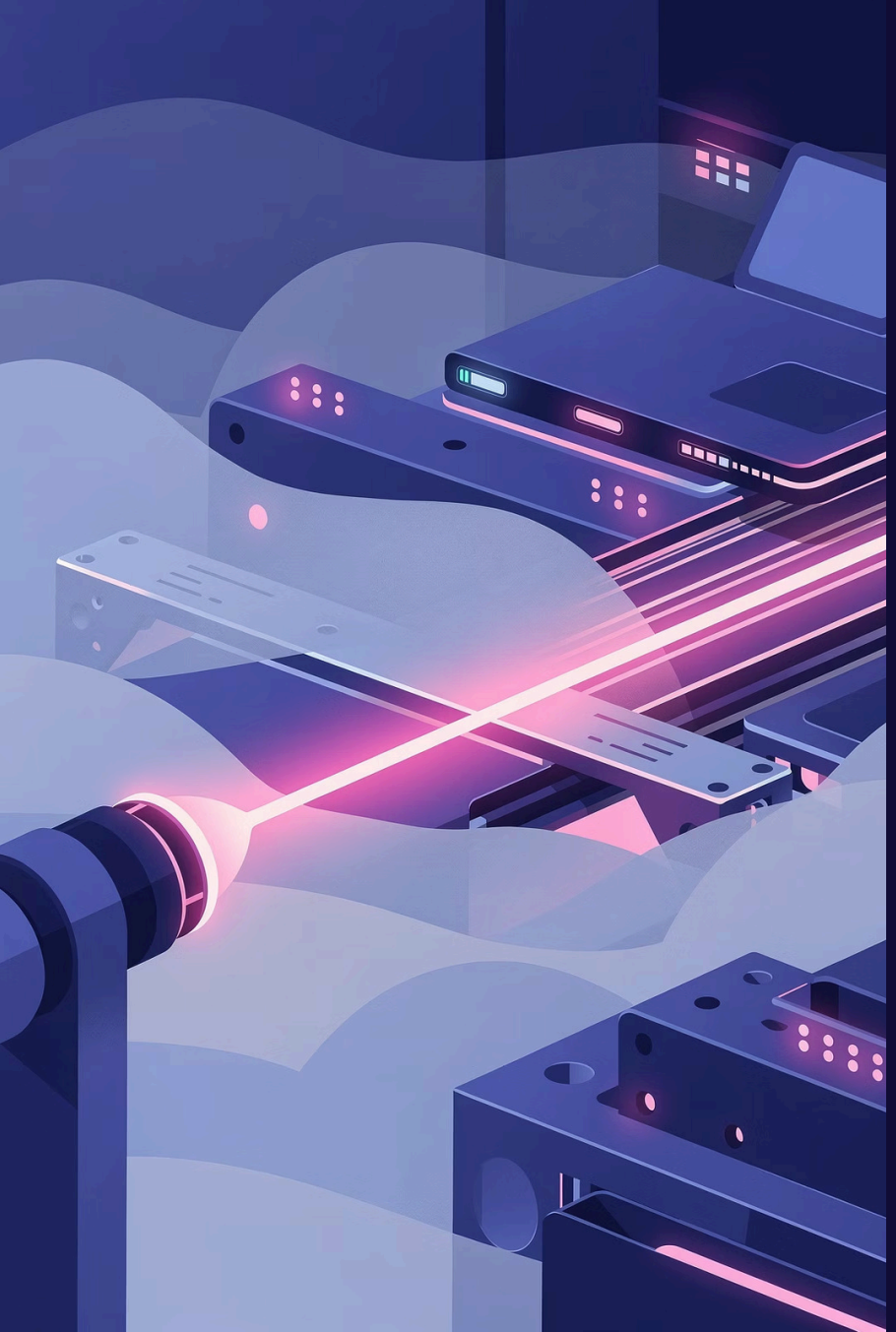
Cannot distinguish between sections. A query about "password policy" returns the entire security guidelines document.

Semantic Dilution

One embedding for 3,000 words averages away the specificity. Dense topics cancel each other out in the embedding space.

Irrelevant Context

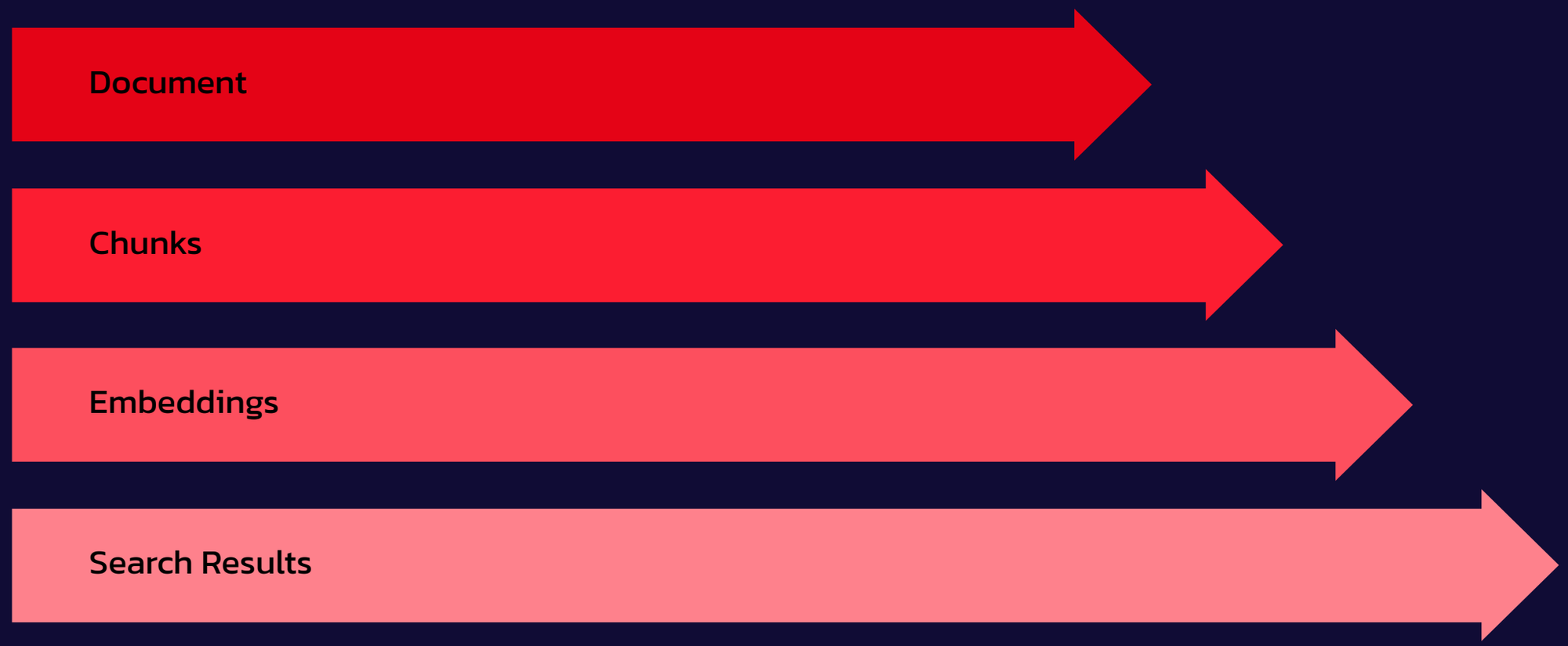
The LLM receives sections with no bearing on the query, increasing hallucination risk and reducing answer quality.



Chunking defines what your system can retrieve.

This is not a configuration detail - it is the most consequential architectural decision in your RAG pipeline. **Choose your chunk size and overlap deliberately.** Everything downstream depends on it.

Fix: Chunking



Chunk Size

Controls how much text lives in a single retrieval unit. Too large → semantic dilution returns. Too small → context fragments lose meaning. Start with **256–512 tokens** for document QA tasks.

Overlap

A sliding window that ensures a sentence split across two chunk boundaries doesn't disappear from retrieval. Typical overlap is **10–20% of chunk size**. Prevents hard boundary artifacts.

What Changes With Chunking?

Moving from full-document to chunked retrieval fundamentally changes the behaviour of every downstream component.

N×

More Vectors

Each document produces multiple index entries — exact count depends on chunk size and overlap settings.

↑

Finer Granularity

Retrieval now surfaces a paragraph, not a page — matching query specificity

↑↑

Higher Precision

Top-K results contain only content directly relevant to the query intent

Live coding check: After chunking, inspect your FAISS index size. It should be significantly larger than the naive version — that's a good sign.

Embeddings Layer

Model: all-MiniLM-L6-v2

A distilled sentence transformer from the `sentence-transformers` library. Optimised for semantic similarity tasks — fast enough for local inference, strong enough for production retrieval.

Library

`sentence-transformers`

Output

384-dimensional dense vectors

Input limit

~256 tokens per passage

Critical Rule

Use the same model for documents and queries.

The embedding space is model-specific. A document vector from MiniLM and a query vector from a different model live in incompatible spaces. Cosine similarity between them is meaningless.

One model. Always.

Vector Search with FAISS

How FAISS Works Here

- Build a flat `IndexFlatIP` - exact inner product search
- Normalise all vectors to unit length before indexing
- Inner product on normalised vectors = cosine similarity
- At query time: embed, normalise, call `index.search(q, k)`

Why This Matters

This is how production vector databases work internally.

Pinecone, Weaviate, Qdrant - they all wrap the same core logic: normalised dense vectors, approximate nearest-neighbour search, top-K return. FAISS gives you the raw mechanics without the abstraction layer.

Retrieval Output

After calling `index.search()`, always inspect what you got back. Never pass raw retrieval output directly to the LLM without a sanity check.

Field	Example Value	Type	Notes
`source`	security_guidelines.txt	string	Which document the chunk came from
`chunk_id`	42	int	Position in the chunked sequence
`score`	0.87	float	Cosine similarity (higher = more relevant)
`text`	"Passwords must be..."	string	Preview of the retrieved chunk content

Always inspect retrieval. If the top-K scores are below 0.5, your chunking strategy or query phrasing needs revisiting — not your LLM.

Grounded Prompt Construction

Prompt Template Pattern

CONTEXT:
{chunk_1}
{chunk_2}
{chunk_3}

QUESTION:
{user_query}

INSTRUCTION:
Answer ONLY using the context above.
If the answer is not in the context,
say "I don't know."

Why This Works

Context First

Retrieved chunks placed before the question prime the model's attention

Hard Constraint

"Answer only from context" is the primary hallucination suppressor

Graceful Fallback

"I don't know" is a valid, honest answer — better than a confident hallucination

Validation – Multi-Query Test

Run all four test queries through the chunked pipeline. Compare output quality against the naive baseline.

1

Reimbursement Policy

"What is the maximum travel reimbursement?" — Tests cross-document disambiguation

2

Onboarding Steps

"What should a new employee do in their first week?" — Tests sequential content retrieval

3

Password Rules

"What are the password complexity requirements?" — Tests specific policy lookup

4

Device Troubleshooting

"How do I reset my work laptop?" — Tests procedural content retrieval

Observation: Chunked retrieval surfaces the *exact paragraph* containing the answer — not the entire document. Precision is the proof.

Key Takeaways + What's Next

Session 7 Learnings

Retrieval > Generation

Answer quality is capped by what you retrieve. Fix retrieval first, always.

Chunking is Critical

The atomic retrieval unit determines everything downstream. Choose deliberately.

Embeddings Power Search

Same model, both sides. Embedding space coherence is non-negotiable.

RAG is Engineering

Not a library call. A system you design, wire, and validate component by component.

Session 8 – Preview

01

FastAPI Integration

Wrap the RAG pipeline in a REST API

02

`/rag/query` Endpoint

Accept a query string, return a grounded JSON response

03

Productionising RAG

Async loading, index persistence, error handling at scale

Next session: your RAG pipeline gets an API.